LABORATORIO NRF_24LO1

Universidad Militar Nueva Granada – Ingeniería en Telecomunicaciones
Miguel Ángel Plazas – Daniel Mateo Alegría Bernate
Docente: José de Jesús Rúgeles Uribe

Resumen – En esta práctica se evaluó el desempeño de un enlace de comunicación inalámbrica basado en módulos nRF24L01+ y placas Raspberry Pi Pico 2W, considerando el intercambio de datos, la confiabilidad de la transmisión y el comportamiento de la señal de radiofrecuencia.

Sobre este enlace se realizaron cuatro análisis. Primero, se analizaron las transacciones SPI mediante un analizador lógico, incluyendo la lectura y escritura de registros y la carga de paquetes, identificando los comandos, los datos intercambiados y los tiempos de transferencia. Posteriormente, se calculó el alcance teórico para diferentes potencias de transmisión y sensibilidades del receptor comparándolos con los resultados de las pruebas de campo. Asimismo, se evaluó la tasa de error de bits durante la transmisión de archivos de texto con alfabetos y datos pseudoaleatorios. Finalmente, se verificó la frecuencia del canal, analizando el ancho de banda y los espectrogramas para comprender la ocupación de la banda de 2,4 GHz y la distribución de canales frente a las redes Wi-Fi.

Palabras clave - Comunicación inalámbrica, nRF24L01+, Raspberry Pi Pico 2W, protocolo SPI, tasa de error de bits, análisis espectral, ancho de banda.

Abstract - In this practical exercise, the performance of a wireless communication link based on nRF24L01+ modules and Raspberry Pi Pico 2W boards was evaluated, with a focus on data exchange, transmission reliability, and radio-frequency signal behavior.

Four types of analysis were conducted on this link. First, SPI transactions were examined using a logic analyzer—including register read/write operations and packet loading—to identify commands, exchanged data, and transfer timings. Subsequently, the theoretical range was calculated for various transmission power levels and receiver sensitivities, and these values were compared against results from field tests. Additionally, the bit error rate was assessed during the transmission of text files containing alphabetic characters and pseudo-random data. Finally, the channel frequency was verified through an analysis of bandwidth and spectrograms, providing insight into the occupancy of the 2.4 GHz band and the distribution of channels in relation to Wi-Fi networks.

Keywords - Wireless communication, nRF24L01+, Raspberry Pi Pico 2W, SPI protocol, bit error rate, spectral analysis, bandwidth.

I. INTRODUCCIÓN

Los sistemas de comunicación digital inalámbrica permiten transmitir información sin una conexión física entre los dispositivos. En la banda ISM de 2,4 GHz, estas comunicaciones deben operar teniendo en cuenta la distribución de canales y la coexistencia con tecnologías como Wi-Fi y Bluetooth. Por ello, el diseño y la evaluación de un enlace inalámbrico requieren analizar parámetros como la modulación, la frecuencia de operación, la potencia de transmisión, la velocidad de datos y la calidad de la comunicación. En esta práctica se implementó un sistema de comunicación con modulación GFSK para estudiar su funcionamiento y evaluar sus características mediante diferentes herramientas de medición.

El sistema se desarrolló utilizando dos placas Raspberry Pi Pico 2W y módulos transceptores nRF24L01+, configurados como transmisor y receptor. Para evaluar su funcionamiento y desempeño, se realizaron cuatro pruebas complementarias. La primera consistió en analizar la comunicación SPI mediante un analizador lógico, registrando operaciones de lectura y escritura de registros, así como la transferencia de paquetes, para identificar los comandos, los datos y los tiempos involucrados. La segunda se centró en calcular el alcance teórico para diferentes potencias de transmisión y sensibilidades del receptor, y comparar los resultados con las mediciones obtenidas en pruebas de campo. La tercera permitió evaluar la tasa de error de bits durante la transmisión de archivos de texto con alfabetos y datos pseudoaleatorios. Finalmente, se examinó el espectro de las señales transmitidas para verificar la frecuencia del canal, estimar el ancho de banda y estudiar la distribución de canales sin traslape, así como la ocupación espectral y la canalización de las redes Wi-Fi en la banda de 2,4 GHz.

II. OBJETIVOS

Objetivo General

Analizar el funcionamiento y el desempeño de un sistema de comunicación digital inalámbrica basado en módulos nRF24L01+, mediante el estudio de la comunicación SPI, el alcance del enlace, la tasa de error de bits y las características espectrales de la señal.

Objetivos específicos

- Comprender el funcionamiento de la interfaz SPI y su relación con la configuración y el control del transceptor nRF24L01+.

- Establecer la relación entre la potencia de transmisión, la sensibilidad del receptor y el alcance del enlace inalámbrico, considerando los resultados teóricos y experimentales.

- Examinar la confiabilidad de la transmisión inalámbrica mediante la evaluación de errores en el envío de diferentes tipos de datos.

    - Caracterizar el comportamiento espectral del sistema para determinar el ancho de banda de la señal, verificar la frecuencia de operación y examinar la distribución de canales en la banda de 2,4 GHz.

III. MARCO TEÓRICO

A. Comunicación inalámbrica en la banda ISM de 2.4 GHz

La banda ISM de 2,4 GHz comprende aproximadamente de 2400 a 2483,5 MHz y es compartida por tecnologías como Wi-Fi, Bluetooth y otros dispositivos de corto alcance. Al ser una banda compartida, los sistemas que operan en ella pueden experimentar interferencias [1]. La frecuencia configurada debe mantenerse dentro de los límites aplicables al servicio y al lugar de operación.

B. Modulación GFSK

Es una técnica de modulación digital que representa la información mediante cambios en la frecuencia de una portadora. Se diferencia de la FSK convencional porque utiliza un filtro gaussiano antes de la modulación, suavizando las transiciones de frecuencia y reduciendo el ancho de banda ocupado. El nRF24L01+ emplea esta técnica para transmitir datos de forma inalámbrica en la banda de 2,4 GHz [1].

A. C. Transceptor nRF24L01+

El nRF24L01+ es un transceptor de radiofrecuencia para la transmisión y recepción de datos. Permite configurar parámetros como el canal, la velocidad de transmisión y la potencia de salida. El registro RF_CH define la frecuencia central mediante la relación indicada en la ecuación (3).

El canal de operación se establece mediante el registro RF_CH, cuyo valor determina la frecuencia central de transmisión según la expresión:

$$f = 2400\,MHz + RF_{CH}\ \ MHz \qquad (1)$$

El valor de RF_CH es un entero entre 0 y 125; según la relación del fabricante.

D. Interfaz de comunicación SPI

SPI es un protocolo serial síncrono y dúplex completo que permite intercambiar información entre un maestro y un esclavo. En esta práctica, la Raspberry Pi Pico 2W actúa como maestro y el nRF24L01+ como esclavo. La interfaz utiliza cuatro señales principales:

- SCK: señal de reloj generada por el maestro.
- MOSI: transmite datos del maestro al esclavo.
- MISO: transmite datos del esclavo al maestro.
- CSN: selecciona el dispositivo y permanece en nivel bajo durante la transacción.

El nRF24L01+ trabaja en modo SPI 0 (CPOL = 0 y CPHA = 0), transmite primero el bit más significativo (MSB) y organiza los datos en bytes de ocho bits [1].

El tiempo ideal necesario para transferir n bytes, cada uno de ocho bits, a una frecuencia de reloj f_SCK se calcula mediante la siguiente ecuación

$$t_{teo} = \frac{(8 \times n)}{f_{sck}} \qquad (2)$$

Para comparar con la medición se usa el error relativo:

$$E\% = \frac{|t_{med} - t_{teo}|}{t_{teo}} \qquad (3)$$

E. Comandos SPI del nRF24L01+

Cada transacción SPI comienza con un byte de comando. Mientras el maestro lo transmite por MOSI, el nRF24L01+ devuelve por MISO el registro STATUS. Después del reinicio, el valor observado para STATUS es 0x0E [1]. Los comandos y registros relevantes se resumen en la Tabla I.

Comando / registro	Valor	Función
R_REGISTER	0x00 | dirección	Lee un registro
W_REGISTER	0x20 | dirección	Escribe un registro
W_TX_PAYLOAD	0xA0	Carga el paquete a transmitir
NOP	0xFF	Sin operación; genera reloj
SETUP_AW	dirección 0x03	Ancho de dirección
RF_CH	dirección 0x05	Canal de RF
STATUS	dirección 0x07	Estado e interrupciones

Tabla 1.Comandos y registros del nRF24L01+.

F. Raspberry Pi Pico

La Raspberry Pi Pico 2W utiliza el microcontrolador RP2350 e incorpora interfaces SPI por hardware y GPIO de 3,3 V, compatibles eléctricamente con el nRF24L01+ [3].

G. Analizador lógico

El analizador lógico registra el nivel digital de varias líneas en el tiempo. Para capturar una señal digital sin perder flancos, la frecuencia de muestreo debe ser varias veces mayor que la frecuencia de la señal. El software asociado puede además decodificar el protocolo y mostrar los bytes transmitidos por MOSI y MISO, junto con el tiempo de cada transacción, lo que permite verificar comandos, datos y señales de control del bus [4].

H. Propagación y alcance del enlace

El alcance teórico de un enlace se estima con un balance de enlace, que compara la potencia que llega al receptor con su sensibilidad. [5]La potencia recibida, en dBm, es:

$$P_{rx} = P_{tx} + G_{tx} + G_{rx} - L_p \qquad (4)$$

En espacio libre, a partir de la ecuación :

$$L = 20\,log_{10}(d) + 20\,log_{10}(f) + 32{,}44 \qquad (5)$$

Para estimar la distancia máxima, se calcula el margen disponible del enlace [5]:

$$d_{max} = 10^{\frac{L - 32{,}44 - 20\,log_{10}(f)}{20}}\ km \qquad (6)$$

I. Tasa de error de bits

Un indicador de la confiabilidad de una comunicación digital y representa la proporción de bits recibidos incorrectamente respecto al total de bits transmitido [6].Se calcula mediante

$$BER = \frac{Ne}{Nt} \qquad (7)$$

Una BER baja indica una transmisión más confiable, mientras que una BER elevada evidencia una mayor cantidad de errores.

J. Análisis de espectro

un instrumento de medición que permite visualizar la distribución de potencia de una señal en función de la frecuencia [7]. Se utiliza para identificar componentes frecuenciales, determinar la frecuencia de operación y observar la ocupación espectral de las señales, facilitando el estudio de posibles interferencias y el solapamiento entre canales de comunicación.

K. Ancho de banda

El intervalo de frecuencias ocupado por una señal y se obtiene mediante la diferencia entre sus frecuencias límite superior e inferior [8]:

$$\Delta f = f2 - f1 \qquad (8)$$

$$BW = 2\Delta f \qquad (9)$$

IV. METODOLOGÍA

A. Construcción del sistema

Se implementó un sistema de comunicación inalámbrica compuesto por dos nodos: un transmisor (TX) y un receptor (RX). Cada nodo utiliza una Raspberry Pi Pico 2W, un transceptor nRF24L01+, una pantalla OLED SSD1306 conectada por I2C, dos pulsadores y tres LED con sus respectivas resistencias limitadoras de corriente.

Los pulsadores se utilizaron para recorrer y confirmar las opciones de configuración del sistema, mientras que los LED permitieron identificar los estados de funcionamiento. La pantalla OLED se empleó para visualizar los parámetros configurados y los mensajes correspondientes a cada nodo.

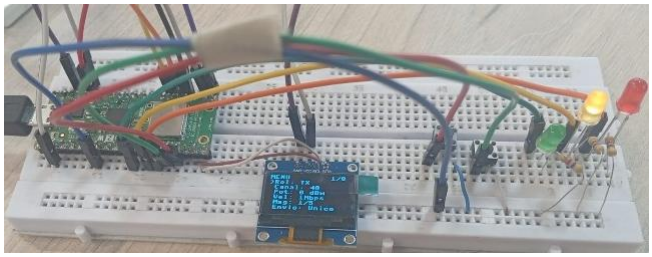


Figure 1 Montaje experimental del transmisor (TX).

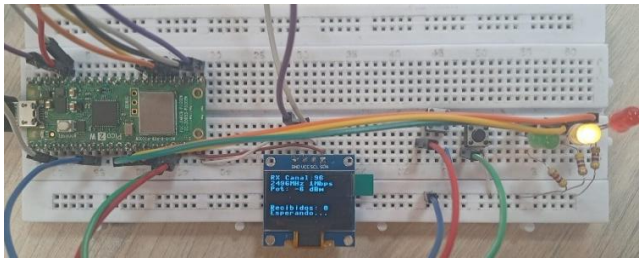


Figure 2.Montaje experimental del receptor (RX).

El análisis experimental se centró en la interfaz SPI entre la Raspberry Pi Pico 2W del transmisor y el nRF24L01+. Se observaron una lectura de registro, una escritura del registro RF_CH y la carga de un mensaje en el transceptor. Estas pruebas permitieron identificar comandos, respuestas y temporización del bus.

B. Conexiones Implementadas

Se conectaron los periféricos a la Raspberry Pi Pico 2W: la pantalla OLED SSD1306, con dirección I2C 0x3C; el transceptor nRF24L01+, mediante SPI; y el analizador lógico, conectado a las señales SPI del nodo transmisor.

Las conexiones son iguales en ambas placas:

Elemento	Señal	Pin Pico
nRF24L01	SCK	GP18
nRF24L01	MOSI	GP19
nRF24L01	MISO	GP16
nRF24L01	CSN	GP17
nRF24L01	CE	GP20
OLED	SDA / SCL	GP14 / GP15
Pulsador variar	entrada	GP13
Pulsador seleccionar	entrada	GP12
LED rojo (error)	salida	GP11
LED amarillo (configurando)	salida	GP10
LED verde (enviado/recibido)	salida	GP9

Tabla 2. Conexiones Raspberry y periféricos.

C. Diseño del software

El firmware, escrito en MicroPython, se divide en dos archivos:

nrf24.py: controlador del módulo. Encapsula las operaciones SPI (lectura y escritura de registros, carga de payload, selección de canal, velocidad y potencia).

main.py: lógica de la aplicación.

El funcionamiento del sistema es el siguiente:

- Arranque: se muestra un mensaje de bienvenida en la OLED, y luego el canal, la velocidad de transmisión y la potencia configurados.
- Configuración: el pulsador «variar» permite recorrer las opciones y «seleccionar» confirma la elección. Se pueden configurar canal, potencia, velocidad y mensaje. Aunque el firmware permite seleccionar los canales disponibles del módulo, los canales por encima de 2483,5 MHz quedan fuera de la banda ISM de 2,4 GHz.
- Mensajes: el TX dispone de una lista con mensajes cortos (nombres de los estudiantes) y largos (alfabetos), además de un modo de envío en bucle.
- Indicación con LEDs: el rojo señala un error, el amarillo indica que el sistema está configurando o esperando, y el verde indica que un mensaje fue enviado o recibido.
- El RX permanece escuchando en el mismo canal y velocidad, y muestra en la OLED el mensaje recibido.

El código del firmware se incluye en el **Anexo A**

C. Configuración del analizador lógico

Para observar la comunicación entre la Raspberry Pi Pico 2W y el nRF24L01+ se conectó un analizador lógico al nodo TX y se utilizó el software Logic 2 [4]. Se emplearon cuatro canales, conectados de la siguiente manera:

Channel 1: SCK (GP18)
Channel 2: MOSI (GP19)
Channel 3: MISO (GP16)
Channel 4: CSN (GP17)

En Logic 2 se habilitó el decodificador SPI y se asignó el canal 4 a CSN, utilizando su flanco de bajada para identificar el inicio de cada transacción. La frecuencia de muestreo se configuró en 12 MS/s, suficiente para observar el reloj SPI de 1 MHz y decodificar los bytes capturados.

V. PRUEBAS Y ANÁLISIS DE RESULTADOS

A. Análisis de SPI

Para cada operación se inició la captura y se ejecutó en el TX la acción correspondiente: reinicio de la placa para observar la lectura de registro, cambio de canal mediante los pulsadores para provocar la escritura y envío del mensaje para capturar la carga del paquete. En Logic 2 se identificaron el comando, los bytes de MOSI y MISO, el comportamiento de CSN y los tiempos registrados. El tiempo teórico se calculó con la ecuación (4) y el error relativo con la ecuación (5).

1. Lectura de registros

Para observar una lectura, se inició la captura del analizador lógico antes de reiniciar el nodo transmisor. Durante el arranque, la Raspberry Pi Pico 2W ejecuta operaciones de configuración y lectura de registros del nRF24L01+. En la captura se seleccionó la transacción correspondiente al registro SETUP_AW.

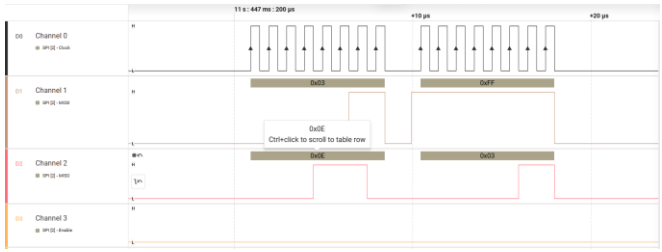


Figure 3. Conexión del analizador lógico al nodo TX.

En la captura se identificó la transacción de lectura y se verificaron las señales CSN, SCK, MOSI y MISO. CSN permanece en nivel bajo durante la transacción y vuelve a nivel alto al finalizarla.

La lectura corresponde al comando R_REGISTER para la dirección del registro SETUP_AW (0x03). El comando se obtiene aplicando la operación OR entre 0x00 y la dirección 0x03:

R_REGISTER = 0x00 OR 0x03 = 0x03

Durante el primer byte, la Pico transmite el comando 0x03 por MOSI y el nRF24L01+ devuelve el byte STATUS por MISO (0x0E en la captura). Para recibir el contenido del registro, el maestro transmite después 0xFF (NOP), que genera los pulsos de reloj; el módulo devuelve 0x03, correspondiente al ancho de dirección configurado de cinco bytes.

Para dos bytes y una frecuencia de reloj f_SCK = 1 MHz, el tiempo ideal calculado mediante la ecuación (2) es:

$$t_{teo} = \frac{8 \times 2}{1\ MHz} = 16\mu s$$

Con el tiempo de reloj activo medido de 17 µs, el error relativo respecto al valor teórico se calcula mediante la ecuación (3):

$$E\% = \frac{|17\ -\ 16|}{16} \times 100 = 6.25\ \%$$

El tiempo durante el cual CSN permanece en bajo es de 101,75 µs, aproximadamente 5,99 veces el tiempo de reloj activo (101,75/17 = 5,99). La diferencia de 84,75 µs entre ambos valores corresponde al tiempo adicional observado entre la activación y desactivación de CSN. Esta duración incluye la gestión de la transacción por el firmware y no debe confundirse con el tiempo durante el cual se generan pulsos de reloj.

2. Escritura que modifica el canal

Para analizar la modificación del canal de radiofrecuencia, se realizó una escritura en el registro RF_CH del nRF24L01+. La operación se generó al modificar el canal mediante el pulsador correspondiente y confirmar la selección.

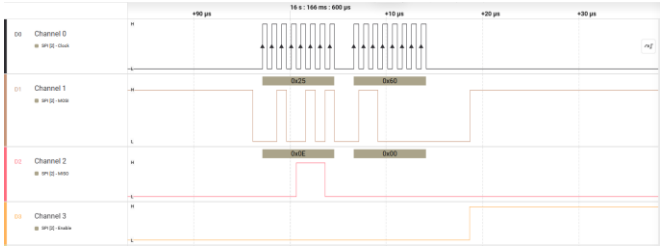


Figure 4.Escritura del registro RF_CH al cambiar el canal.

En la captura se identifican los bytes 0x25 y 0x60. El primero es el comando W_REGISTER combinado con la dirección del registro RF_CH: 0x20 OR 0x05 = 0x25. El segundo byte, 0x60, equivale a 96 en decimal y configura el canal 96. La frecuencia correspondiente se calcula con la ecuación (1):

$$f\ =\ 2400\ MHz\ +\ 96\ =\ 2496\ MHz$$

Por MISO se recibe 0x0E durante el byte de comando y 0x00 durante el byte de datos. En una escritura, este segundo byte no representa una lectura del contenido de RF_CH. CSN pasa a nivel bajo antes de los pulsos de SCK y vuelve a nivel alto al finalizar la transacción.

Medida	**Valor**
Duración de un byte	7.583 µs
Inicio byte 1 a inicio byte 2	9.5 µs
Pausa entre bytes	2 µs
Reloj activo (2 bytes)	17.0 µs
CSN en bajo	75,833 µs

Tabla 3. Tiempos medidos en la escritura del registro

Para dos bytes y f_SCK = 1 MHz, el tiempo ideal calculado mediante la ecuación (4) es:

$$t_{teo} = \frac{8 \times 2}{1\ MHz} = 16\mu s$$

Con un tiempo de reloj activo medido de 17 µs, el error relativo respecto al valor teórico es:

$$E\% = \frac{|17\ -\ 16|}{16} \times 100 = 6.25\ \%$$

La diferencia de 1 µs entre el tiempo ideal (16 µs) y el tiempo de reloj activo medido (17 µs) equivale a un error relativo del 6,25 %. En esta captura, CSN permanece en bajo durante 75,833 µs, es decir, unas 4,46 veces el tiempo de reloj activo (75,833/17 = 4,46). La diferencia de 58,833 µs corresponde al tiempo adicional de la transacción, asociado al control de CSN y a la ejecución del firmware.

3. Transferencia de un paquete hacia el transceptor

Para analizar la transferencia de información hacia el nRF24L01+, se envió un mensaje desde el nodo transmisor y se localizó la transacción correspondiente al comando W_TX_PAYLOAD (0xA0), utilizado para cargar el paquete que posteriormente podrá transmitirse por radio.

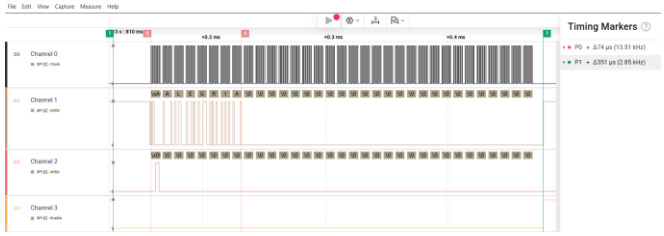


Figure 5. Vista general

Como mensaje de prueba se utilizó la palabra ALEGRIA, formada por siete caracteres. En la tabla se presenta el código hexadecimal correspondiente a cada carácter:

Letra	**A**	**L**	**E**	**G**	**R**	**I**	**A**
Hex	0x41	0x4C	0x45	0x47	0x52	0x49	0x41

Tabla 4 . Códigos ASCII mensaje ALEGRIA.

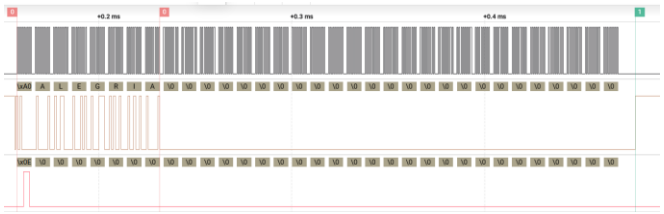


Figure 6. Envío de Mensaje (Alegría)

El tiempo de reloj activo desde el comando 0xA0 hasta el último carácter del mensaje fue de 74,0 µs. La transferencia analizada contiene ocho bytes: el byte de comando y los siete caracteres de ALEGRIA. Por tanto, el tiempo ideal de reloj, para n = 8 y f_SCK = 1 MHz, se obtiene con la ecuación (2):

$$t_{teo} = \frac{8 \times 8}{1\ MHz} = 64\ \mu s$$

Con el tiempo de reloj activo medido de 74,0 µs, el error relativo respecto al tiempo teórico se calcula mediante la ecuación (5):

$$E\% = \frac{|74\ -\ 64|}{64} \times 100 = 15.6\ \%$$

La diferencia entre los 74,0 µs medidos y los 64 µs ideales se relaciona con los intervalos entre bytes observados en la captura. Si los bytes comienzan cada 9,5 µs, los ocho bytes ocupan aproximadamente 9,5 × 7 + 7,583 = 74,083 µs. Después de ALEGRIA, la transacción continúa con bytes de relleno 0x00; estos no se incluyen en la medición de los ocho bytes analizados.

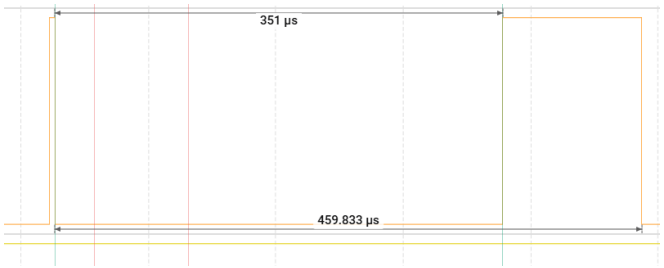


Figure 7. Vista del CSn

Después de ALEGRIA, la captura muestra 25 bytes de relleno 0x00, de modo que la transacción completa tiene 33 bytes: el comando y 32 bytes de datos, que es la longitud máxima del payload del módulo. Con los bytes iniciándose cada 9,5 µs, el reloj permanece activo durante aproximadamente 9,5 × 32 + 7,583 = 311,6 µs. CSN permanece en bajo durante 351 µs, por lo que el tiempo adicional atribuible al firmware es de unos 351 − 311,6 = 39 µs.

B. Análisis de alcance

El alcance teórico se estimó con el balance de enlace del marco teórico para las cuatro potencias de transmisión programables (−18, −12, −6 y 0 dBm), las velocidades de 1 y 2 Mbps y los dos niveles de ganancia del LNA, a la frecuencia del canal 96 (2496 MHz). La velocidad de 250 kbps no se evaluó porque uno de los módulos no cuenta con ese modo; el firmware lo detecta en soporta_250k(), que escribe el bit RF_DR_LOW y lo lee de vuelta, por lo que la lista VELS queda limitada a 1Mbps y 2Mbps.

1. Amplificador de bajo ruido (LNA) y sensibilidad

El LNA (Low Noise Amplifier) es la primera etapa del receptor. Amplifica la señal captada por la antena, que llega con muy poca potencia, agregando el menor ruido posible para no degradar la relación señal a ruido de las etapas siguientes. En el nRF24L01 su ganancia tiene dos niveles, controlados por el bit LNA_HCURR del registro RF_SETUP: con ganancia alta el receptor logra su máxima sensibilidad, y con ganancia baja se reduce el consumo en 0,8 mA a costa de perder 1,5 dB de sensibilidad [1]. En el firmware, set_lna() escribe este bit y sens_dbm() suma los 1,5 dB cuando el LNA es bajo. La sensibilidad es la potencia mínima con la que el receptor opera con un BER menor al 0,1 %; el fabricante especifica −85 dBm a 1 Mbps y −82 dBm a 2 Mbps [1]. Con LNA bajo estos valores pasan a −83,5 y −80,5 dBm, respectivamente.

2. Cálculo del alcance teórico

Para que el receptor pueda demodular la señal, la potencia recibida debe ser al menos igual a su sensibilidad S. Al igualar la ecuación (4) con S, la pérdida máxima de trayecto admisible es:

$$d = 10^{\frac{L-32{,}44-20\log(Mhz)}{20}}$$

Se consideró una ganancia nominal de 2 dBi para cada antena (parámetro ANT_DBI del firmware). La distancia máxima se obtiene al reemplazar Lmax y f = 2496 MHz en la ecuación (6). Como ejemplo, para 1 Mbps, LNA alto y 0 dBm:

$$d = 10^{\frac{89-32{,}44-20\log(2496)}{20}}$$

$$d = 270m$$

Para 2 Mbps, LNA bajo y −18 dBm, siguiendo el ejemplo anteiro se obtuvo d = 20 m. Este cálculo está implementado en presupuesto_db() y alcance_m(), y la opción «Alcance teórico» del menú (pantalla_alcance()) lo muestra en la OLED para la configuración activa. De las ecuaciones se desprende que cada 6 dB menos de potencia reducen el alcance a la mitad, que el LNA bajo (−1,5 dB) lo disminuye cerca de un 16 % y que pasar de 1 a 2 Mbps (−3 dB) lo disminuye cerca de un 29 %.

3. Pruebas de campo

Las pruebas se realizaron al aire libre, con el RX fijo y el TX alejándose progresivamente, en el canal 96. Para cada combinación de potencia, velocidad y LNA se registró la distancia máxima a la que el RX seguía recibiendo los mensajes. La distancia se estimó contando pasos, con 1 paso = 1 m, por lo que los resultados tienen una exactitud limitada, sobre todo en distancias largas.

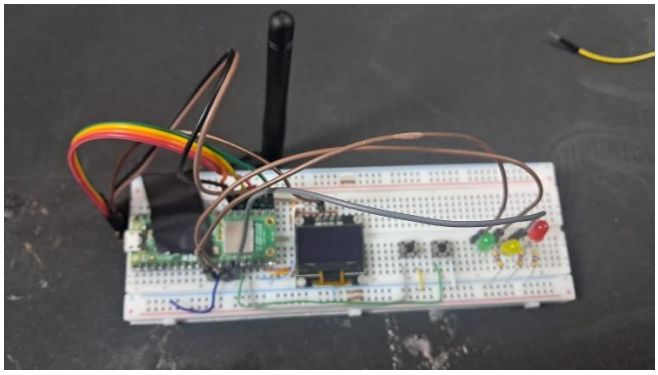


Figure 8. Montaje Experimental

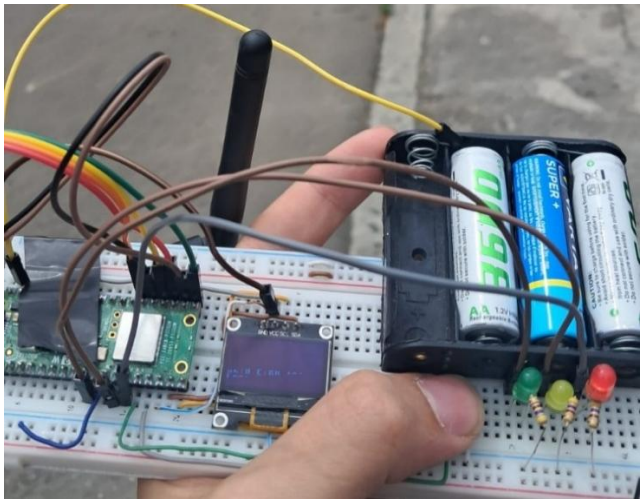


Figure 9. Montaje Experimental

La Tabla 5 compara los alcances teóricos con los medidos. El error relativo se calculó como:

Tabla 5. Alcance teórico y experimental (canal 96, G = 2 dBi por antena).

Vel	LNA	Sens (dBm)	Pot (dBm)	L (dB)	Teo (m)	Exp (m)	%e
1Mbps	Alta	-85	0	89	270	140	48,1
1Mbps	Alta	-85	-6	83	135	80	40,7
1Mbps	Alta	-85	-12	77	68	44	35,3
1Mbps	Alta	-85	-18	71	34	22	35,3
1Mbps	Baja	-83,5	0	87,5	227	140	38,3
1Mbps	Baja	-83,5	-6	84,5	114	75	34,2
1Mbps	Baja	-83,5	-12	75,5	57	37	35,1
1Mbps	Baja	-83,5	-18	69,5	29	18	38
2Mbps	Alta	-82	0	86	191	125	34,6
2Mbps	Alta	-82	-6	80	96	62	34,4
2Mbps	Alta	-82	-12	74	48	31	35,4
2Mbps	Alta	-82	-18	68	24	15	37,5
2Mbps	Baja	-80,5	0	84,5	161	105	34,8
2Mbps	Baja	-80,5	-6	78,5	80	52	35
2Mbps	Baja	-80,5	-12	72,5	40	26	35
2Mbps	Baja	-80,5	-18	66,5	20	10	50

C. Análisis de errores

1. Procedimiento de medición

Para medir el BER se transmitieron desde el TX hasta el RX dos archivos de texto de 1792 bytes (N_ARCHIVO): alfabetos.txt, con la secuencia de 62 caracteres A–Z, a–z y 0–9 repetida (gen_alfabetos()), y aleatorio.txt, con caracteres ASCII imprimibles pseudoaleatorios generados con un congruencial lineal de semilla fija (gen_aleatorio()). Así, las dos Pico producen el mismo contenido y el RX conoce los datos esperados.

Cada archivo se divide en 64 paquetes de 28 bytes de datos, a los que armar_trama() antepone un encabezado de 4 bytes (número de secuencia de 2 bytes y su complemento) para completar los 32 bytes del payload. El archivo se envía de 1 a 20 veces (PASADAS), hasta 1280 paquetes y 286 720 bits de datos, con una pausa de 15 ms entre paquetes (PAUSA_BER_MS). Al final se envía cinco veces una trama de fin con el total de paquetes y el checksum del archivo (armar_fin()). Para que el hardware no corrija ni descarte los paquetes con errores, el módulo se configuró sin ACK y sin CRC, aplicar_radio (ack=False, crc=False).

En el RX, ber_procesar() verifica el encabezado con su complemento: si está dañado, el paquete se cuenta en hdr_malos y se descarta; si es válido, compara los 28 bytes recibidos con el trozo esperado (chunk()), cuenta los bits distintos aplicando la tabla POP al resultado de la XOR y calcula el BER con la ecuación (7), donde Ne son los bits erróneos y Nt los bits comparados. Los resultados se guardan en resultados_ber.csv (guardar_resultado()) junto con la tasa de error de paquetes (PER).



Figure 10. Configuración de la prueba de BER.

El campo chk del archivo indica que el checksum del archivo coincidió entre TX y RX en las diez pruebas. Como el BER solo considera los paquetes con encabezado íntegro, no incluye los paquetes perdidos; por eso se reporta junto con el PER. Todas las pruebas se hicieron en el canal 96 (2496 MHz), con LNA alto y desde la misma distancia ,para las dos velocidades y los dos archivos, con potencias entre −18 y −6 dBn. Como el CSV no registra la potencia, los resultados se agruparon en dos series según el orden de adquisición.

2. Resultados

Tabla 6. Resultados de la prueba de BER (serie 1)

Serie 1: Ptx=-18dBm								
N	Archivo	Vel.	Paq. Tx	Paq. Rx	Hdr. Dañ	Bits Err	BER	PER
1	Alfabeto	2Mbps	320	160	13	202	5,60E-03	50
2	Alfabeto	1Mbps	1280	1009	45	745	3,3 E-3	21,2
3	Aleatorio	1Mbps	1280	1064	66	742	3,1 E-3	16,9
4	Alfabeto	2Mbps	1328	630	61	1118	7,9 E-3	52,6
5	Aleatorio	2Mbps	1280	315	79	1579	2,2 E-2	75,4

Tabla 6. Resultados de la prueba de BER (serie 2)

Serie 2: Ptx=-12dBm								
N	Archivo	Vel.	Paq. Tx	Paq. Rx	Hdr. Dañ	Bits Err	BER	PER
6	Alfabeto	1Mbps	1280	1278	0	7	2,40E-05	0,2
7	Alfabeto	1Mbps	1280	1279	0	11	3,80E-05	0,1
8	Aleatorio	1Mbps	128/0	1245	8	85	3,10E-04	2,7
9	Alfabeto	2Mbps	1280	1279	0	1	3,50E-06	0,1
10	Aleatorio	2Mbps	1280	1242	4	153	5,50E-04	3

D. Análisis de espectro

En esta sección se analiza el espectro del sistema inalámbrico mediante el analizador Anritsu MS2724B, con el fin de verificar la frecuencia del canal seleccionado y comparar el comportamiento de la señal en dos modalidades de transmisión.

1. Montaje

Para realizar las pruebas de comunicación inalámbrica, se implementó un montaje compuesto por dos módulos nRF24L01, uno configurado como transmisor (TX) y otro como receptor (RX) dispuestos en estaciones separadas donde se realizaría sus procesos de recepción y transmisión.



Figure 11 Montaje experimental para el análisis de espectro.

Para el análisis espectral, se utilizó el analizador de espectros Anritsu MS2724B, con el fin de observar las emisiones de radiofrecuencia y verificar la frecuencia de operación del canal configurado.

a. Canalización

Para seleccionar el canal de transmisión del nRF24L01, se analizó la banda de 2,4 GHz mediante el analizador de espectros para identificar las frecuencias ocupadas por otras señales inalámbricas y reducir posibles interferencias

Durante la observación se encontró actividad entre 2,460 y 2,475 GHz, compatible con emisiones de redes Wi-Fi de los canales 11 al 13. Por esta razón, se eligió el **canal 96** del nRF24L01**,** cuya frecuencia central teórica es de 2496 MHz, ubicada por encima del límite habitual de la banda Wi-Fi de 2,4 GHz (2483,5 MHz). Esta separación permite reducir la superposición con las señales observadas y disponer de una frecuencia de operación diferenciada.

4. Configuración

El módulo nRF24L01 se configuró en el canal 96, con una potencia de transmisión de −6 dBm. Para evaluar la señal en diferentes condiciones, se implementaron dos modalidades de envío en el código:

Transmisión intermitente: El transmisor enviaba un mensaje cada 100 ms. Considerando un tiempo aproximado de transmisión de 329 µs por paquete, la emisión ocupaba una pequeña fracción del intervalo entre mensajes.

Por esta razón, al realizar un barrido de 379 ms, el analizador registró ráfagas aisladas de energía, que aparecen como puntos dispersos en el espectrograma, debido a que la señal está presente durante periodos breves y permanece ausente durante la mayor parte del tiempo entre transmisiones.

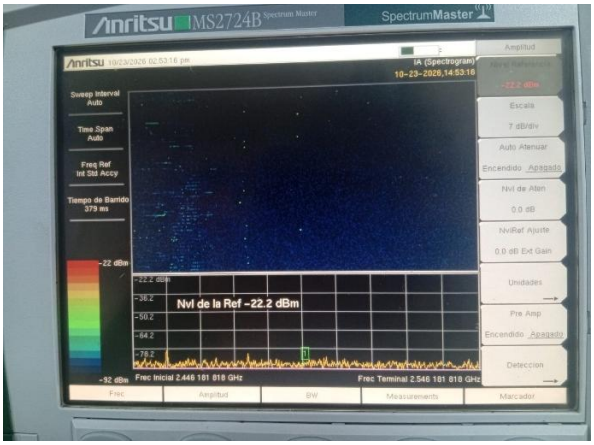


Figure 12. Espectrograma de la transmisión intermitente.

Debido a la baja persistencia de la señal en esta primera modalidad, se implementó una segunda configuración en la que los paquetes se enviaban consecutivamente. Al reducir los intervalos entre transmisiones, aumentó la ocupación temporal del canal y la señal permaneció visible durante una mayor parte del barrido.

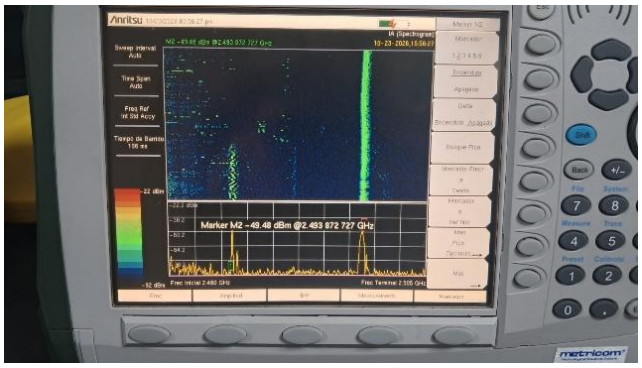


Figure 13. Espectrograma de la transmisión consecutiva de paquetes.

5. Frecuencia esperada

La frecuencia de operación del nRF24L01 se determina mediante el valor programado en el registro RF_CH. Para este dispositivo, la frecuencia central se calcula mediante la expresión:

$$fcanal = 2400\,MHz + RF_CH\,MHz$$

Sustituyendo RF_CH = 96

$$fcanal = 2400 + 96 = 2496\,MHz$$

Por lo tanto, la frecuencia central teórica del canal seleccionado es de 2,496 GHz. A partir de las mediciones realizadas con el analizador, se obtuvieron los siguientes resultados:

Transmisión	Frecuencia	E%
Intermitente	2496,3636 MHz	-0,014
Consecutiva	2493,2873 MHz	0,11

Tabla 5 Resultados frecuencias

Los resultados muestran diferencias tanto en la frecuencia registrada como en el nivel de potencia indicado por el analizador. En la transmisión intermitente, la frecuencia medida fue de 2496,3636 MHz, un valor cercano a la frecuencia teórica de 2496 MHz. En cambio, durante la transmisión consecutiva, la frecuencia registrada fue de 2493,2873 MHz, aproximadamente 2,713 MHz por debajo del valor esperado.

a. Ancho de banda

Para determinar el ancho de banda de la señal transmitida, se tomó como referencia el pico del espectro, cuya potencia aproximada fue de −49.52 dBm. A partir de este valor, se estableció un nivel 20 dB inferior, correspondiente a −69.52 dBm, para identificar la separación en frecuencia entre el pico y uno de los extremos de la señal.



Figure 14. Señal registrada canal 96

Las frecuencias registradas fueron:

- Frecuencia del pico: 2.493.218.181 MHz.
- Frecuencia del punto a −20 dB respecto al pico: 2.493.954.545 MHz

Primero, se calculó la separación entre ambos puntos:

$$\Delta f = f2 - f1$$

$$\Delta f = 2{,}493.954.545\ \text{MHz} - 2{,}493.218.181\ \text{MHz}$$

$$\Delta f = 0{,}736364\ \text{MHz}$$

Considerando que el espectro presenta una distribución aproximadamente simétrica respecto al pico, se estimó el ancho de banda completo duplicando esta separación:

$$BW = 2\Delta f$$

$$BW = 2 \times 0{,}736364\ \text{MHz} = 1{,}472728\ \text{MHz}$$

Por lo tanto, el ancho de banda estimado de la señal es de 1,473 MHz

4. ANÁLISIS DE RESULTADOS

1. Comparación de las mediciones experimentales

A continuación, se resumen los resultados obtenidos durante las pruebas de lectura de registros, escritura del canal de radiofrecuencia y carga de un paquete de datos en el transceptor nRF24L01+.

Operación	Datos identificados	Bytes	Tiempo teórico	Tiempo medido	Error relativo
Lectura de registros	R_REGISTER (0x03+ 0xFF)	2	16 µs	17 µs	6,25%
Escritura del canal	0x25, 0x60	2	16 µs	17 µs	6,25%
Carga del paquete	0xA0 +ALEGRIA	8	64 µs	74 µs	15,6 %

Tabla 6. Comparación entre los tiempos teóricos y medidos en las tres operaciones SPI.

Las diferencias entre los tiempos teóricos y experimentales se explican principalmente por la temporización de la comunicación SPI. En la lectura y escritura del canal, la diferencia fue pequeña de 16 a 17 µs, mientras que en la carga de «ALEGRIA» el intervalo aumentó de 64 a 74 µs. Esto ocurre porque el cálculo teórico considera una transferencia continua a 1 MHz, mientras que la ejecución real puede incluir pausas entre bytes debido a la secuencia del programa. Por tanto, el tiempo adicional no implica necesariamente un error en la comunicación.

Además, CSN permaneció en nivel bajo durante más tiempo que la transferencia de datos, ya que delimita la operación completa y no únicamente los pulsos de reloj.

2. Análisis alcance real vs experimental

Los alcances experimentales siguen la tendencia prevista por el modelo teórico (Fig. 10). Al reducir la potencia de transmisión de 0 a −18 dBm, el alcance a 1 Mbps con el LNA en nivel alto disminuyó de 140 a 22 m. Asimismo, al reducir la potencia en intervalos de 6 dB, el alcance se aproximó a la mitad en varias configuraciones, en concordancia con la relación establecida en la ecuación (6). El cambio a LNA bajo y el aumento de la velocidad a 2 Mbps también redujeron el alcance. Por ejemplo, a −12 dBm se midieron 44 m a 1 Mbps con LNA alto, 37 m a 1 Mbps con LNA bajo, 31 m a 2 Mbps con LNA alto y 26 m a 2 Mbps con LNA bajo. Este comportamiento se explica por la menor sensibilidad efectiva del receptor al reducir la ganancia del LNA y al aumentar la velocidad de transmisión, condiciones que disminuyen el margen disponible para mantener el enlace.

A 0 dBm el LNA bajo no redujo el alcance medido (140 m en ambos casos), lo que se atribuye a la incertidumbre de la medición, pues a esa distancia el 16 % equivale a unos 20 m. Sin embargo, en todos los casos el alcance medido fue menor al teórico, y el error se mantuvo entre −34 % y −38 % en 14 de las 16 configuraciones. Un error casi constante indica una diferencia fija en el presupuesto de enlace: una reducción del 35 % equivale a unos 4 dB, justo la suma de las ganancias supuestas para las dos antenas. De hecho, al calcular el alcance sin ganancia de antena se obtienen 170, 85, 43 y 21 m para 1 Mbps con LNA alto, valores que coinciden con los medidos a −6, −12 y −18 dBm (80, 44 y 22 m).

Esto sugiere que la ganancia efectiva de las antenas fue cercana a 0 dBi y no a los 2 dBi nominales, por la eficiencia de las antenas de los módulos, el desacople de polarización o la cercanía al suelo. Las mayores desviaciones (−48 % y −51 %) ocurren en la distancia más larga y en la más corta de la prueba: en la primera influyen obstáculos y reflexiones en el suelo, y en la segunda un paso de diferencia representa un 10 % de la distancia.

3. Análisis tasa de error

Efecto del nivel de señal. En la Serie 1 el enlace estuvo cerca de su límite de sensibilidad: el BER fue de $3{,}1 \times 10^{-3}$ a $2{,}2 \times 10^{-2}$, por encima del 10^{-3} con que el fabricante define la sensibilidad, y se perdió entre el 17 % y el 75 % de los paquetes. En la Serie 2 el BER bajó a valores entre $3{,}5 \times 10^{-6}$ y $5{,}5 \times 10^{-4}$, inferiores a 10^{-3}, y el PER a menos del 3 %. Entre series el BER disminuyó entre 10 y 2000 veces según la prueba, lo que muestra que cuando el margen del enlace es pequeño una mejora de las condiciones de recepción cambia el desempeño de forma muy marcada.

Efecto de la velocidad. En la Serie 1, 2 Mbps presentó más errores que 1 Mbps: $5{,}6 \times 10^{-3}$ y $7{,}9 \times 10^{-3}$ frente a $3{,}3 \times 10^{-3}$ con alfabetos, y $2{,}2 \times 10^{-3}$ frente a $3{,}1 \times 10^{-3}$ con datos aleatorios, además de un PER mayor (53 % y 75 % frente a 21 % y 17 %), lo cual es coherente con la sensibilidad 3 dB menor a 2 Mbps. En la Serie 2 la tendencia se mantiene con datos aleatorios ($5{,}5 \times 10^{-4}$ frente a 3×10^{-4}); con alfabetos, 2 Mbps tuvo un solo bit erróneo en 286 496 bits ($3{,}5 \times 10^{-6}$, la mínima resolución de la prueba), una cantidad tan baja que su comparación con 1 Mbps (7 y 11 bits erróneos) tiene poca validez.

Efecto del tipo de archivo. En la Serie 2 los datos aleatorios dieron un BER mayor que los alfabetos ($3{,}0 \times 10^{-4}$ frente a

$2{,}4 \times 10^{-5}$ y $3{,}8 \times 10^{-5}$ a 1 Mbps $5{,}5 \times 10^{-4}$; frente a $3{,}5 \times 10^{-6}$ a 2 Mbps) y fueron los únicos con encabezados dañados (8 y 4 paquetes). Una posible explicación es que el alfabeto es un patrón periódico de 62 símbolos, mientras que los datos pseudoaleatorios tienen secuencias de bits más variadas y el filtro gaussiano del GFSK introduce más interferencia entre símbolos. Sin embargo, a 1 Mbps en la Serie 1 los dos archivos dieron un BER similar ($3{,}3 \times 10^{-3}$ y $3{,}1 \times 10^{-3}$) y cada caso se midió una sola vez, por lo que no puede concluirse que el tipo de archivo sea la causa.

Paquetes perdidos. El PER es mucho mayor que la cantidad de encabezados dañados: en la última prueba de la Serie 1 se enviaron 1280 paquetes, se recibieron 315 y solo 79 tuvieron el encabezado dañado, por lo que unos 886 nunca llegaron a la aplicación. Como el módulo solo entrega un paquete si reconoce su dirección de 5 bytes, es probable que estas pérdidas se deban a errores en el preámbulo o la dirección, y no solo en los datos.

Trama de fin. En la prueba 4 el total transmitido aparece como 1328, mayor al máximo de 1280 paquetes. La trama de fin también se envía sin CRC, y 1280 (0x0500) y 1328 (0x0530) difieren en dos bits del byte bajo, por lo que el total recibido estaba dañado. Con 1280 el PER sería 50,8 % en lugar de 52,6 %.

4. Análisis de la variación de frecuencia

El cambio fundamental al eliminar el retardo de 100 ms no es solo que se envíen más datos, sino que el transceptor pasa de un modo pulsado (ráfagas) a un régimen de operación continuo y de carga máxima (100% de *Duty Cycle*). Este cambio radical en la dinámica del chip provoca el desplazamiento de 2 MHz (de 2.496 GHz a 2.494 GHz) por los siguientes motivos físicos y de circuitería interna:

- **Modo intermitente (100 ms):** El chip pasa la mayor parte del tiempo en reposo, encendiendo el sintetizador de radiofrecuencia y el PLL únicamente durante el breve instante en que debe enviar el paquete. Al terminar, el circuito se desengancha y descansa

- Modo continuo (Instantáneo): El PLL y el oscilador operan de forma ininterrumpida, sin pausas para reajustarse. Al someter al bucle de enganche de fase a una operación continua sin descanso, el punto de equilibrio del sintetizador puede sufrir una ligera desviación en sus divisores de frecuencia, estabilizándose en un estado ligeramente distinto al del modo pulsado.

5. CONCLUSIONES

1.Funcionamiento de la interfaz SPI: Se verificó la comunicación entre la Raspberry Pi Pico 2W y el transceptor nRF24L01+ mediante la identificación de los comandos, los datos intercambiados y las señales de control CSN, SCK, MOSI y MISO. Las capturas permitieron comprobar la lectura del registro SETUP_AW, la escritura del canal RF_CH y la carga del mensaje ALEGRIA. Además, las diferencias entre los tiempos teóricos y medidos demostraron que la duración real de una transacción depende tanto de los pulsos de reloj como de las pausas introducidas durante la ejecución del firmware.

2.Relación entre potencia, sensibilidad y alcance: Se comprobó experimentalmente que el alcance disminuye al reducir la potencia de transmisión, disminuir la sensibilidad efectiva del receptor o aumentar la velocidad de transmisión. Por ejemplo, con una velocidad de 1 Mbps y LNA alto, el alcance pasó de 140 m a 22 m al reducir la potencia de 0 a −18 dBm. Aunque los resultados siguieron la tendencia prevista por el modelo teórico, las distancias experimentales fueron generalmente inferiores a las calculadas. Esta diferencia evidencia que el balance de enlace ideal no representa por completo las condiciones reales de propagación, en las que también influyen las antenas, los obstáculos, las reflexiones y la incertidumbre de la medición.

3.Confiabilidad de la transmisión de datos: La evaluación del BER y del PER permitió determinar que la calidad del enlace depende de las condiciones de recepción. En la primera serie, realizada a −18 dBm, se obtuvieron BER entre ($3{,}3 \times 10^{-3}$ y $2{,}2 \times 10^{-2}$, junto con pérdidas de paquetes de hasta aproximadamente el 75 %. En la segunda serie, a −12 dBm, los errores y las pérdidas disminuyeron considerablemente. Asimismo, las pruebas a 2 Mbps mostraron un desempeño menos favorable en varios casos que las realizadas a 1 Mbps.

Confirmando que una mayor velocidad puede reducir el margen disponible para la recepción confiable. Sin embargo, las diferencias entre las series deben interpretarse considerando las condiciones de cada prueba y la necesidad de repetir las mediciones. También se evidenció que el BER no representa por sí solo toda la confiabilidad del sistema, ya que no contabiliza los paquetes perdidos; por ello, resulta necesario analizarlo junto con el PER y emplear mecanismos como CRC, confirmaciones y retransmisiones.

4.Caracterización espectral de la señal: El análisis con el analizador de espectros permitió observar las emisiones del nRF24L01+ en transmisiones intermitentes y consecutivas, identificar diferencias en su ocupación temporal y estimar un ancho de banda aproximado de 1,473 MHz a partir del criterio de −20 dB. También se observó que la frecuencia registrada durante la transmisión intermitente estuvo próxima al valor teórico del canal configurado, mientras que la medición de la transmisión consecutiva presentó una diferencia mayor. Esto evidencia que el comportamiento observado depende tanto de la configuración del canal como de la forma en que se transmiten los datos.

6. REFERENCIAS

[1] Nordic Semiconductor, nRF24L01+ Single Chip 2.4GHz Transceiver, Product Specification v1.0, 2008. [Online]. Disponible en https://cdn.sparkfun.com/assets/home_ page_ poSts/2/6/7/3/nRF24L01P.pdf.

[2] MicroPython, “class SPI – a Serial Peripheral Interface bus protocol (controller side),” MicroPython Documentation, 2026. [Online]. Disponible en: https://docs.micropython.org/ en/latest/library/machine.SPI.html.

[3] Raspberry Pi Ltd., Raspberry Pi Pico 2 W Datasheet, 2026. [Online]. Disponible en: https://datasheets.raspberrypi.com/ picow/pico-2-w-datasheet.pdf.

[4] Saleae, “SPI Analyzer – User Guide,” Saleae Support, 2026. [Online]. Disponible en: https://support.saleae.com/ protocol-analyzers/ analyzer-user-guides/using-spi.

[5] International Telecommunication Union, Recommendation ITU-R P.525-4: Calculation of Free-Space Attenuation. Geneva, Switzerland: ITU, 2019. Disponible en: https://www.itu.int/rec/R-REC-P.525/en

[6] B. Sklar, Digital Communications: Fundamentals and Applications, 2nd ed. Upper Saddle River, NJ, USA: Prentice Hall, 2001.

[7] Keysight Technologies, Spectrum Analysis Basics. Disponible en: https://www.keysight.com/

7. ANEXOS

ANEXO A.

A1. Código main.py

```
#main.py
from machine import Pin, SPI, I2C
import ssd1306, time
from nrf24 import NRF24

# ---------- Hardware ----------
i2c = I2C(1, sda=Pin(14), scl=Pin(15), freq=400000)
oled = ssd1306.SSD1306_I2C(128, 64, i2c, addr=0x3C)
spi = SPI(0, baudrate=1000000, polarity=0, phase=0,
sck=Pin(18), mosi=Pin(19), miso=Pin(16))
nrf = NRF24(spi, csn=Pin(17, Pin.OUT), ce=Pin(20,
Pin.OUT))

b_var = Pin(13, Pin.IN, Pin.PULL_UP)     # variar entre
opciones
b_sel = Pin(12, Pin.IN, Pin.PULL_UP)     # seleccionar
led_r = Pin(11, Pin.OUT)              # rojo: error / no se
transmitio
led_a = Pin(10, Pin.OUT)              # amarillo: configurando /
esperando
led_v = Pin(9, Pin.OUT)               # verde: enviado / recibido

# ---------- Valores definidos ----------
CANAL_MAX = 125                       # canales 0..125 ->
2400..2525 MHz
POT_TXT = ["-18 dBm", "-12 dBm", "-6 dBm", "0 dBm"]
VELOCIDADES = ["250K", "1M", "2M"]
VEL_TXT = ["250Kbps", "1Mbps", "2Mbps"]
ROLES = ["TX", "RX"]
ENVIOS = ["Unico", "Loop 1 msg", "Loop lista"]     # modo
de envio
INTERVALOS = [100, 250, 500, 1000, 2000]          # ms
entre envios del loop
IV_TXT = ["100 ms", "250 ms", "500 ms", "1 s", "2 s"]
MENSAJES = [b"ALEGRIA", b"MIGUELITO",
b"ABCDEFGHIJKLMNOPQRSTUVWXYZ",
b"abcdefghijklmnopqrstuvwxyz",
b"ABCDEFGHIJKLMNOPQRSTUVWXYZ123456"]   #
maximo 32 bytes

# configuracion actual ("can" es el numero de canal
directamente: 0..125)
cfg = {"rol": 0, "can": 40, "pot": 3, "vel": 1, "msg": 0, "env":
0, "iv": 3}

# ---------- Utilidades ----------
def leds(r=0, a=0, v=0):
led_r(r)
led_a(a)
led_v(v)

def leer_botones():
"""Devuelve 'var', 'var_largo', 'sel' o None."""
if b_var.value() == 0:
time.sleep_ms(30)
t0 = time.ticks_ms()
while b_var.value() == 0 and time.ticks_diff(time.ticks_ms(),
t0) < 800:
time.sleep_ms(10)
largo = b_var.value() == 0
while b_var.value() == 0:
time.sleep_ms(10)
return "var_largo" if largo else "var"
if b_sel.value() == 0:
time.sleep_ms(30)
while b_sel.value() == 0:
time.sleep_ms(10)
return "sel"
return None

def esperar_boton():
ev = None
while ev is None:
ev = leer_botones()
time.sleep_ms(10)
return ev

def hay_boton():
```

```
return b_var.value() == 0 or b_sel.value() == 0

def esperar_soltar():
while hay_boton():
time.sleep_ms(10)
time.sleep_ms(30)

def aplicar():
"""Carga la configuracion elegida en el NRF24L01."""
nrf.ce(0)
nrf.set_channel(cfg["can"])
nrf.set_power(cfg["pot"])
nrf.set_datarate(VELOCIDADES[cfg["vel"]])
if cfg["rol"] == 1:
nrf.start_rx()

# ---------- Pantallas ----------
def verificar_nrf():
if nrf.read_reg(0x03) != 0x03:       # SETUP_AW debe leerse
como 0x03
leds(r=1)
oled.fill(0)
oled.text("ERROR NRF24L01", 0, 20)
oled.text("Revise cableado", 0, 32)
oled.show()
while True:
time.sleep(1)

def inicio():
leds(a=1)
oled.fill(0)
oled.text("Comunicacion", 0, 8)
oled.text("digital", 0, 20)
oled.text("inalambrica", 0, 32)
oled.text("GFSK - UMNG", 0, 48)
oled.show()
time.sleep(2.5)
verificar_nrf()
canal = cfg["can"]
oled.fill(0)
oled.text("CONFIGURACION", 0, 0)
oled.text("Canal: %d" % canal, 0, 14)
oled.text("Freq: %d MHz" % (2400 + canal), 0, 26)
oled.text("Vel: " + VEL_TXT[cfg["vel"]], 0, 38)
oled.text("Pot: " + POT_TXT[cfg["pot"]], 0, 50)
oled.show()
time.sleep(3)

def filas_menu():
return ["Rol: " + ROLES[cfg["rol"]],
"Canal: %d" % cfg["can"],
"Pot: " + POT_TXT[cfg["pot"]],
"Vel: " + VEL_TXT[cfg["vel"]],
"Msg: %d/%d" % (cfg["msg"] + 1, len(MENSAJES)),
"Envio: " + ENVIOS[cfg["env"]],
"Intervalo: " + IV_TXT[cfg["iv"]],
"Iniciar"]

def dibujar_menu(sel):
filas = filas_menu()
ini = max(0, sel - 5)              # ventana de 6 filas con
desplazamiento
oled.fill(0)
oled.text("MENU", 0, 0)
oled.text("%d/%d" % (sel + 1, len(filas)), 96, 0)
for k in range(6):
i = ini + k
if i >= len(filas):
break
oled.text((">" if i == sel else " ") + filas[i], 0, 9 + 9 * k)
oled.show()

def elegir(titulo, opciones, idx):
"""opciones: lista de (linea1, linea2). GP13 cambia, GP12
acepta."""
while True:
oled.fill(0)
oled.text(titulo, 0, 0)
l1, l2 = opciones[idx]
oled.text(l1, 0, 20)
oled.text(l2, 0, 32)
oled.text("13: cambiar", 0, 46)
oled.text("12: aceptar", 0, 55)
oled.show()
ev = esperar_boton()
if ev in ("var", "var_largo"):
idx = (idx + 1) % len(opciones)
elif ev == "sel":
return idx

def elegir_canal(ch):
"""Canal 0..125. GP13 corto: +1, GP13 largo: +10, GP12:
aceptar."""
while True:
oled.fill(0)
oled.text("CANAL", 0, 0)
oled.text("Canal: %d" % ch, 0, 16)
oled.text("Freq: %d MHz" % (2400 + ch), 0, 28)
oled.text("13:+1 largo:+10", 0, 44)
oled.text("12: aceptar", 0, 55)
oled.show()
ev = esperar_boton()
if ev == "var":
ch = (ch + 1) % (CANAL_MAX + 1)
elif ev == "var_largo":
ch = (ch + 10) % (CANAL_MAX + 1)
elif ev == "sel":
return ch

def menu():
leds(a=1)                          # amarillo: configurando
sel = 0
n = len(filas_menu())
while True:
dibujar_menu(sel)
ev = esperar_boton()
```

```
if ev in ("var", "var_largo"):
sel = (sel + 1) % n
elif ev == "sel":
if sel == 0:
cfg["rol"] = elegir("ROL", [("TRANSMISOR", ""),
("RECEPTOR", "")], cfg["rol"])
elif sel == 1:
cfg["can"] = elegir_canal(cfg["can"])
elif sel == 2:
cfg["pot"] = elegir("POTENCIA TX", [(p, "") for p in
POT_TXT], cfg["pot"])
elif sel == 3:
cfg["vel"] = elegir("VELOCIDAD", [(v, "") for v in
VEL_TXT], cfg["vel"])
elif sel == 4:
cfg["msg"] = elegir("MENSAJE",
[(m.decode()[:16], m.decode()[16:32]) for m in MENSAJES],
cfg["msg"])
elif sel == 5:
cfg["env"] = elegir("MODO DE ENVIO", [(e, "") for e in
ENVIOS], cfg["env"])
elif sel == 6:
cfg["iv"] = elegir("INTERVALO LOOP", [(t, "") for t in
IV_TXT], cfg["iv"])
else:
return

def pantalla_run(titulo, texto, extra, estado):
canal = cfg["can"]
oled.fill(0)
oled.text("%s Canal:%d" % (titulo, canal), 0, 0)
oled.text("%dMHz %s" % (2400 + canal,
VEL_TXT[cfg["vel"]]), 0, 9)
oled.text("Pot: " + POT_TXT[cfg["pot"]], 0, 18)
oled.text(texto[:16], 0, 30)
oled.text(texto[16:32], 0, 39)
oled.text(extra[:16], 0, 48)
oled.text(estado[:16], 0, 56)
oled.show()

# ---------- Modo transmisor ----------
def enviar_uno(msg, cont):
"""Envia un mensaje, actualiza contadores y LEDs. Devuelve
True si hubo ACK."""
leds(a=1)
ok = nrf.send(msg)
print("TX:", msg, "OK" if ok else "SIN ACK")
if ok:
cont["ok"] += 1
leds(v=1)
else:
cont["err"] += 1
leds(r=1)
return ok

def loop_tx(cont):
"""Envia en bucle hasta que se presione cualquier boton."""
espera = INTERVALOS[cfg["iv"]]
i = cfg["msg"]
n = 0
while True:
msg = MENSAJES[i] if cfg["env"] == 2 else
MENSAJES[cfg["msg"]]
n += 1
ok = enviar_uno(msg, cont)
pantalla_run("TX", msg.decode(), "OK:%d Err:%d" %
(cont["ok"], cont["err"]),
"Loop #%d" % n)
t0 = time.ticks_ms()
while time.ticks_diff(time.ticks_ms(), t0) < espera:
if hay_boton():             # cualquier boton detiene el loop
esperar_soltar()
leds(a=1)
return
time.sleep_ms(5)
if cfg["env"] == 2:
i = (i + 1) % len(MENSAJES)

def correr_tx():
aplicar()
cont = {"ok": 0, "err": 0}
estado = "12:Enviar" if cfg["env"] == 0 else "12:Iniciar loop"
leds(a=1)
while True:
msg = MENSAJES[cfg["msg"]]
pantalla_run("TX", msg.decode(), "OK:%d Err:%d" %
(cont["ok"], cont["err"]), estado)
ev = esperar_boton()
if ev == "var":                         # siguiente mensaje
cfg["msg"] = (cfg["msg"] + 1) % len(MENSAJES)
estado = "Msg %d/%d" % (cfg["msg"] + 1, len(MENSAJES))
elif ev == "var_largo":                 # volver al menu
return
elif ev == "sel":
if cfg["env"] == 0:                     # envio unico
pantalla_run("TX", msg.decode(),
"OK:%d Err:%d" % (cont["ok"], cont["err"]), "Enviando...")
ok = enviar_uno(msg, cont)
estado = "Enviado" if ok else "ERROR sin ACK"
pantalla_run("TX", msg.decode(),
"OK:%d Err:%d" % (cont["ok"], cont["err"]), estado)
time.sleep_ms(800)
leds(a=1)
else:                                   # loop
loop_tx(cont)
estado = "Loop detenido"

# ---------- Modo receptor ----------
def correr_rx():
aplicar()
n = 0
texto = ""
estado = "Esperando..."
t_led = 0
leds(a=1)
pantalla_run("RX", texto, "Recibidos: %d" % n, estado)
```

```
while True:
if nrf.available():
data = nrf.recv().rstrip(b'\x00')
texto = data.decode('utf-8', 'ignore')
n += 1
print("RX:", texto)
leds(v=1)
t_led = time.ticks_add(time.ticks_ms(), 600)
estado = "Recibido"
pantalla_run("RX", texto, "Recibidos: %d" % n, estado)
if t_led and time.ticks_diff(time.ticks_ms(), t_led) >= 0:
leds(a=1)
t_led = 0
ev = leer_botones()
if ev == "var_largo":                    # volver al menu
return
if ev == "sel":                          # reiniciar contador
n = 0
texto = ""
estado = "Esperando..."
pantalla_run("RX", texto, "Recibidos: %d" % n, estado)
time.sleep_ms(10)

# ---------- Programa principal ----------
inicio()
while True:
menu()
if cfg["rol"] == 0:
correr_tx()
else:
correr_rx()
```

A2. Código nrf24

```
#nrf24.py
import time

class NRF24:
   def __init__(self, spi, csn, ce, payload=32,
addr=b"UMNG1"):
      self.spi, self.csn, self.ce = spi, csn, ce
      self.payload = payload
      self.cfg = 0x0C                # CRC activado, 2 bytes
      self.csn(1)
      self.ce(0)
      time.sleep_ms(100)
      self.write_reg(0x00, self.cfg)    # CONFIG
      self.write_reg(0x01, 0x01)        # auto-ACK en pipe 0
      self.write_reg(0x02, 0x01)        # habilitar pipe 0
      self.write_reg(0x03, 0x03)        # direcciones de 5 bytes
      self.write_reg(0x04, 0x2F)        # 15 reintentos, espera
750 us
      self.write_reg(0x05, 40)          # canal inicial
      self.write_reg(0x06, 0x06)        # 1 Mbps, 0 dBm
      self.write_reg(0x10, addr)        # TX_ADDR
      self.write_reg(0x0A, addr)        # RX_ADDR_P0
      self.write_reg(0x11, payload)     # tamano del payload
      self.cmd(0xE1)                 # flush TX
      self.cmd(0xE2)                 # flush RX
      self.write_reg(0x07, 0x70)        # limpiar banderas

   # ----- SPI basico -----
   def cmd(self, c):
      self.csn(0)
      self.spi.write(bytes([c]))
      self.csn(1)

   def write_reg(self, reg, val):
      if isinstance(val, int):
         val = bytes([val])
      self.csn(0)
      self.spi.write(bytes([0x20 | reg]) + val)
      self.csn(1)

   def read_reg(self, reg):
      rx = bytearray(2)
      self.csn(0)
      self.spi.write_readinto(bytes([reg, 0xFF]), rx)
      self.csn(1)
      return rx[1]

   def status(self):
      rx = bytearray(1)
      self.csn(0)
      self.spi.write_readinto(b'\xff', rx)
      self.csn(1)
      return rx[0]

   # ----- Configuracion -----
   def set_channel(self, ch):
      # RF_CH admite 0..125  ->  f = 2400 + ch (MHz)  ->
2400 a 2525 MHz
      ch = max(0, min(125, int(ch)))
      self.write_reg(0x05, ch)

   def get_channel(self):
      return self.read_reg(0x05)

   def set_power(self, level):          # 0=-18, 1=-12, 2=-6, 3=0
dBm
      rf = self.read_reg(0x06) & 0xF9
      self.write_reg(0x06, rf | (level << 1))

   def set_datarate(self, vel):         # "250K", "1M" o "2M"
      rf = self.read_reg(0x06) & 0xD7
      rf |= {"250K": 0x20, "1M": 0x00, "2M": 0x08}[vel]
      self.write_reg(0x06, rf)

   # ----- Transmisor -----
   def send(self, data, timeout_ms=300):
      data = bytes(data)[:self.payload]
      data = data + bytes(self.payload - len(data))
      self.ce(0)
```

```
        self.write_reg(0x00, self.cfg | 0x02)      # PWR_UP,
modo TX
        time.sleep_ms(2)
        self.cmd(0xE1)
        self.write_reg(0x07, 0x70)
        self.csn(0)
        self.spi.write(b'\xA0' + data)             #
W_TX_PAYLOAD
        self.csn(1)
        self.ce(1)
        time.sleep_us(20)
        self.ce(0)
        t0 = time.ticks_ms()
        while time.ticks_diff(time.ticks_ms(), t0) < timeout_ms:
            st = self.status()
            if st & 0x20:                          # TX_DS: enviado con
ACK
                self.write_reg(0x07, 0x20)
                return True
            if st & 0x10:                          # MAX_RT: sin ACK
                self.write_reg(0x07, 0x10)
                self.cmd(0xE1)
                return False
        self.cmd(0xE1)
        return False

    # ----- Receptor -----
    def start_rx(self):
        self.ce(0)
        self.write_reg(0x00, self.cfg | 0x03)      # PWR_UP,
modo RX
        time.sleep_ms(5)
        self.write_reg(0x07, 0x70)
        self.cmd(0xE2)
        self.ce(1)
        time.sleep_us(150)

    def available(self):
        return not (self.read_reg(0x17) & 0x01)    #
FIFO_STATUS: RX no vacio

    def recv(self):
        rx = bytearray(self.payload + 1)
        self.csn(0)
        self.spi.write_readinto(b'\x61' + bytes(self.payload), rx)
# R_RX_PAYLOAD
        self.csn(1)
        self.write_reg(0x07, 0x40)
        return bytes(rx[1:])
```